

# Architectural Incorrigoibility and Conditional Liveness in Ledger-Governed Agents

September 19, 2026

## Abstract

An AI agent usually depends on an operator who can alter its rules or stop its execution. We study an architecture in which a blockchain stores the agent’s state and pays replaceable workers to run its program. Verified outputs can authorize updates, alone or together with designated governors. We characterize *architectural incorrigibility*: which groups can change a protected part of the agent without its approval. This requires examining sequences of rule changes, since a restriction may be bypassed by changing who enforces it. The same approval requirements that prevent intervention can also block repair. For execution, we bound the probability of continued failure when workers remain available and affordable, each retains a minimum chance of success, and obstruction has a minimum unrecoverable cost. With a finite attack budget and continuing attempts, this bound tends to zero. This does not establish perpetual operation from a finite fund. Hardware attestation, recent proofs of language-model computation, and a source-level case study illustrate the analysis. The central distinction is between who may intervene and what resources and cooperation are needed to keep running.

## 1 Introduction

An AI agent commonly inherits its operator’s shutdown switch: control over the process, credentials, or payment account is sufficient to stop it. A blockchain wallet alone leaves the computation dependent on that operator. A different construction stores the agent’s persistent state, execution rules, and budget in *smart contracts*, programs whose rules the blockchain enforces. Independent workers run its specified program, including inference by a large language model (LLM), and submit proposed ledger updates with evidence of correct computation. The contracts verify this evidence, enforce their authorization rules, and pay for accepted execution. A worker may disappear while the state, funds, and opportunity for another worker to execute remain available.

Consider an agent managing a fund. Routine transfers might require a verified output of the agent’s program, while replacing its model might require approval from a designated wallet, which we call a *governor*. Moving the entire fund could require both. With several governors, the contract could require the agent and a threshold of their approvals. These are ordinary smart-contract authorization rules with an additional source of approval: verifiable computation in place of a signature from the agent. They can be configured separately for each operation. At an extreme, no governor coalition has an authorized route to replace the program or deactivate the contract without the agent’s consent.

We use *architectural incorrigibility* for resistance to specified interventions enforced by this allocation of authority. In the AI safety literature, corrigibility concerns an agent’s willingness and incentives to accept correction, including shutdown and modification [10, 17]. An agent can be behaviorally cooperative yet lack an authorized shutdown mechanism; conversely, an uncooperative model can sit behind a governor-controlled pause function. Architectural permissions and behavioral dispositions are distinct. Requiring the agent’s consent also has a practical cost: if its execution mechanism fails, that same requirement may prevent the repair needed to obtain its consent.

The paper develops three consequences of this design. First, intervention authority includes every permitted sequence of governance changes, not just the permissions visible now. Second, the groups whose approval can authorize a repair also determine who can block it by withholding approval. Third, if available workers remain affordable and each obstruction incurs at least a fixed unrecoverable loss, a finite attack budget buys only finitely many such delays. A continuing minimum success chance on the remaining attempts then

gives a quantitative recovery guarantee. These are applications of access-control reachability and elementary probability, rather than new cryptographic primitives. Section 2 gives the architecture, Sections 3–4 develop the results, and Sections 5–6 examine their implementation.

## 2 A ledger-governed agent

The basic interface is familiar from smart contracts: someone proposes an operation, the contract checks whether it is authorized, and execution updates ledger state. A conventional contract might accept a transfer only from its owner or from a multisignature wallet. Here it can also accept evidence that an approved agent program produced that transfer. We first separate the proposed update from its authorization, then describe how computation supplies an approval.

### 2.1 Ledger state and proposed updates

An agent might propose transferring funds, recording data, or replacing its program. Describing such a change and obtaining permission to make it are separate steps. Let  $s \in \mathcal{S}$  denote the relevant ledger state, including application data, spendable funding  $B$ , the agent specification  $\theta$ , the authorization policy  $\rho$ , the verification configuration  $\nu$ , and execution-request records. These are application-level components, independent of a particular virtual machine.

An update  $u \in \mathcal{U}$  is an encoded instruction describing one or more proposed changes to ledger state. It might transfer assets, update stored data, or replace a program identifier. Its meaning is given by a partial deterministic function

$$\text{Apply} : \mathcal{S} \times \mathcal{U} \rightarrow \mathcal{S}.$$

The function is *partial* because an update may be inapplicable to the current state, such as a transfer exceeding the available balance. Authorization is a separate check. In an Ethereum implementation,  $u$  could be a contract call or batch of calls; the called code determines the effects, ordering, and failure behavior. The model therefore describes proposed state changes without granting arbitrary write access or prescribing who may authorize them.

The agent’s output has the form  $y = (u, z)$ , where  $u$  is the proposed update and  $z$  is optional auxiliary data, possibly empty. The authorization interface reads the structured update, not an interpretation of the auxiliary data. If some data is itself to be stored on the ledger, that storage operation belongs in  $u$ . A no-op is also an admissible update.

### 2.2 Verifiable computation as approval

A worker must show that it ran the approved model and surrounding program, rather than submit an answer of its own choosing. The *committed specification*  $\theta$  binds the components that determine the output: model weights, prompt construction, arithmetic, decoding, and parsing into the structured update  $u$ . A cryptographic hash can identify those components without storing the full model on the ledger.

An execution request fixes a canonical input  $x$  from an authenticated ledger snapshot and permitted external data, and fixes randomness  $r$  if required. The relation  $R_\theta$  states whether  $y$  is an allowed output of this computation on inputs  $x, r$ :

$$R_\theta(x, r, y) = 1. \tag{1}$$

It may be functional,  $y = F_\theta(x, r)$ . If several outputs satisfy the relation, a worker may select among them without violating soundness; fixed randomness prevents repeated sampling only when the remaining computation is also fixed.

A worker submits  $y$  with evidence  $\pi$ . The contract checks the request and verifies

$$\text{Verify}_\nu(\theta, H(x), r, H(y), \pi) = 1,$$

where  $H$  is a binding hash commitment. Evidence can be a report from protected hardware (*attestation*) or a cryptographic proof of computation [9]. A sound verifier thereby establishes *agent approval* of  $u$  under the specified program and request. For a governor-proposed change, the request can ask the agent to accept

or reject that exact change; the program returns the corresponding update or a no-op. The worker supplies evidence of the decision and cannot substitute a different one.

The input and evidence also bind the chain, contract, request identifier (nonce), and policy version. This prevents an approval for one context from being reused in another. Structured approvals can use EIP-712 on Ethereum; the application must additionally reject reuse of a consumed nonce [4]. The contract checks state preconditions at settlement, so a computation on a prior snapshot does not silently overwrite intervening updates.

## 2.3 Governors and authorization policies

A *governor* is an account whose authenticated approval carries a permission. The basic case is a wallet controlled by a signing key; contract wallets can express multisignature or other governance rules [7, 12]. We identify governors with accounts, rather than with the humans or organizations using them. An approval may arrive as a signature over the proposal or as an authenticated contract call.

Let  $\mathcal{G} = \{G_1, \dots, G_n\}$  be the governors and let  $A$  denote the agent’s computation-based approval. For a given update and request, write  $g_i = 1$  when  $G_i$  has authenticated its approval, and  $a = 1$  when the contract has verified the agent’s approval. An authorization policy is then a Boolean predicate

$$\text{Auth}_\rho(s, u; a, g_1, \dots, g_n).$$

For example, a bilateral treasury migration requires  $a \wedge g_1$ . The same contract might allow routine transfers on  $a$  alone. This extends familiar owner, role, and multisignature checks with a verification predicate; the agent need not hold a signing key. Once authorization and state preconditions hold, the contract applies  $u$ , with request consumption and worker settlement handled by the execution protocol.

For comparing policies, we use the approval sources  $\mathcal{P} = \{A\} \cup \mathcal{G}$ . The symbol  $A$  denotes a verified decision source, not a key that can sign arbitrary proposals. Workers are executors unless separately assigned a governor role. A multisignature governor can be expanded into its underlying signers when its internal threshold matters.

## 2.4 Security assumptions

We assume a secure ledger: finalized state is consistent, and transactions execute the deployed contract rules correctly. We also assume unforgeable account authentication, binding commitments, and sound computation evidence for the specified verification mechanism, or *backend*. Soundness means that invalid computations cannot obtain accepted evidence, except with the backend’s stated failure probability. The relevant hardware or proof-system assumptions are discussed in Section 5. Governors may exercise any authority their accounts possess; their benevolence is not assumed.

Changing which program or verifier the contract accepts is itself an authorized state transition, analyzed in Section 3. Such a change can remove a protection without forging a signature or defeating the original verifier. With the intended verifier in place, accepted evidence establishes that the specified computation produced the update; it does not establish that the update is desirable. Ledger inclusion, available computation, and adequate funding are separate conditions for liveness, stated in Section 4.

# 3 Intervention authority as a graded property

An agent may control its spending while a governor controls its program. Requiring both parties to approve a program change gives each a veto; requiring the agent and two of three governors distributes that authority differently again. To compare these arrangements, we ask which groups’ approvals are needed for each intervention, including changes to the permissions themselves. A governor who can grant itself a role may have more authority than the current role assignments suggest.

## 3.1 Direct authorization

These policies combine governor signatures with verified agent outputs. As before,  $a$  and  $g_i$  denote approval by the agent and  $G_i$ ;  $\wedge$  means “and” and  $\vee$  means “or.”

| Policy                 | Approval formula               | Interpretation               |
|------------------------|--------------------------------|------------------------------|
| Agent unilateral       | $a$                            | Agent can authorize alone    |
| Governor unilateral    | $g_1$                          | Governor can authorize alone |
| Alternative unilateral | $a \vee g_1$                   | Either can authorize alone   |
| Bilateral              | $a \wedge g_1$                 | Both must approve            |
| Multilateral           | $a \wedge [\sum_i g_i \geq k]$ | Agent and $k$ governors      |
| Disabled               | $\perp$                        | No approvals suffice         |

These policies apply separately to each operation. Agent-authorized spending can coexist with governor-authorized upgrades. Permission to act and protection against modification are therefore different properties.

We express these checks using standard *monotone access structures* [3]. For update  $u$  at state  $s$ , let  $\Gamma_u(s) \subseteq 2^{\mathcal{P}}$  be the family of sufficient approval sets, where  $2^{\mathcal{P}}$  denotes all subsets of the approval sources. We restrict attention to policies in which additional approvals can be ignored: if  $C \in \Gamma_u(s)$  and  $C \subseteq D$ , then  $D \in \Gamma_u(s)$ . For example, under a bilateral rule,  $\{A, G_1\}$  suffices, as does  $\{A, G_1, G_2\}$ . The empty family describes a disabled operation; membership of the empty set describes a permissionless one. State and timing preconditions are separate.

To discuss protection, we must also say what is protected. Changing a model, overwriting application state, and disabling execution are different interventions.

**Definition 1** (Intervention target). *An intervention target  $J \subseteq \mathcal{S}$  is the set of states in which a specified intervention has occurred, with  $s_0 \notin J$ . A history-dependent target can be represented by adding a flag or other record to the state.*

For example,  $J$  can record replacement of the approved weights hash, or transfer of a protected reserve outside its mandate. It should specify a technical condition: a changed weights hash is observable, whereas whether the model’s “goals” changed may be unclear. We distinguish *deactivation*, a protocol state preventing execution, from *dormancy*, an interval without successful execution.

### 3.2 Authority through sequences of updates

Suppose the fund’s stored records can be overwritten only with two governors’ approval, but  $G_1$  alone can install a new storage module whose initialization accepts arbitrary records. The direct two-party rule can then be bypassed. There are two routes to the same target:

$$\begin{aligned}
s_0 &\xrightarrow{\{G_1, G_2\}} s_J && \text{(direct overwrite),} \\
s_0 &\xrightarrow{\{G_1\}} s_1 \xrightarrow{\emptyset} s_J && \text{(replace, then initialize).}
\end{aligned}$$

The labels list the approvals needed at each step. The first route needs both governors; the second needs only  $G_1$ , because initialization is permissionless. Collecting all labels along a route therefore exposes authority missed by a direct permission table. This is the familiar access-control safety question of what authority can be acquired through administrative commands [11].

Formally, use a labeled transition graph from  $s_0$ . Each edge  $t$  represents a possible ledger transition, with its current authorization policy and its other preconditions. The graph includes ownership, program, verifier, and delegation changes, and the policies that result from them. External inputs and environmental events are fixed or represented explicitly. An exact graph retains computation feasibility; an abstraction that admits extra transitions provides only an upper bound on reachable interventions.

**Which agent’s approval?** For each analysis, fix the *reference agent approval relation*  $R_A$  at  $s_0$ , including the designated specification and request-binding rules. A label  $A$  records a required fresh approval under that relation. It cannot be supplied merely by choosing to include  $A$  in a coalition: the reference computation must actually produce it. Crucially, replacing the program or verifier does not automatically transfer this label to the replacement. If  $G_1$  installs a program that always approves migration, running that program is a computational precondition, but supplies no reference-agent approval. That route has labels  $\{G_1\}$  and  $\emptyset$ , even if the replacement’s execution is proved correctly. The same convention applies to a verifier that

accepts arbitrary outputs. An agent-approved replacement already puts  $A$  in the path; a separate analysis after an upgrade can choose a new reference agent.

Labels likewise attribute delegated signing authority to its controlling source. Existing approvals are part of  $s_0$ ; the analysis counts fresh approvals needed from that point. For each edge  $t$ ,  $\Gamma_t$  uses this fixed-source attribution, rather than simply copying the mutable contract checks summarized by  $\Gamma_u(s)$ . Its inclusion-minimal members,  $\min \Gamma_t$ , are sufficient sets with no unnecessary approval source. Choose one such set  $L_t$  at each step of a path  $p$ . Its *support*,  $\bigcup_{t \in p} L_t$ , is the set of approval sources required anywhere on that path. Define the effective approval family

$$\Gamma_J^*(s_0) = \left\{ C \subseteq \mathcal{P} : \begin{array}{l} \text{some finite path } p \text{ from } s_0 \text{ reaches } J, \\ \text{with } L_t \in \min \Gamma_t \text{ and } \bigcup_{t \in p} L_t \subseteq C \end{array} \right\}. \quad (2)$$

In words, a set belongs to this family if it contains all required approval sources for at least one route to the target. This includes both routes in the storage example, while identifying  $\{G_1\}$  as the smaller sufficient set.

**Proposition 1** (Authorization reachability). *For fixed  $C \subseteq \mathcal{P}$ , retain each transition alternative whose required approval set is contained in  $C$ , leaving its other preconditions unchanged. Then  $J$  is reachable in this restricted graph if and only if  $C \in \Gamma_J^*(s_0)$ .*

*Proof.* Every path in the restricted graph has all its labels contained in  $C$ , so their union is contained in  $C$ . Conversely, if a path’s union is contained in  $C$ , each of its labels is contained in  $C$ , and every step is retained. In both directions the policy used at a step is the one reached by the preceding steps, including any governance changes.  $\square$

The substantive task is constructing the graph faithfully, especially its upgrade paths and approval labels. The proposition characterizes required approvals; it does not say a governor can compel the reference agent to approve. It is also existential reachability, not an ability to force the same outcome against every transaction ordering. For a finite explicit graph, minimal supports can be computed by propagating label unions and discarding supersets at each state.

### 3.3 Comparing protection against intervention

For each protected feature, we can now list the groups of governors whose intervention paths require no fresh reference-agent approval. Removing groups from a list strengthens protection against that intervention. Keeping a separate list for each feature makes the proposed gradient precise.

**Definition 2** (Architectural incorrigibility profile). *For targets  $J_1, \dots, J_q$ , define the external intervention profile*

$$\mathcal{I}(s_0) = (\Gamma_{J_j}^*(s_0) \cap 2^{\mathcal{G}})_{j=1}^q.$$

*The system is architecturally incorrigible against  $C \subseteq \mathcal{G}$  for  $J_j$  when  $C$  is absent from that component. For the same approval sources, reference agent, targets, and security assumptions, profile  $\mathcal{I}_X$  is at least as incorrigible as  $\mathcal{I}_Y$  when every component of  $\mathcal{I}_X$  is a subset of the corresponding component of  $\mathcal{I}_Y$ .*

An empty component means that no governor-only path reaches that target. It does not say that an agent-approved intervention is possible: a bilateral rule and a disabled operation can both have this external property. The full family  $\Gamma_J^*$  additionally records feasible routes requiring agent approval. Conversely, a governor may persuade the agent to approve an intervention. That path contains a reference-agent approval and so is outside the governor-only family; the profile measures architectural requirements, not behavioral resistance to persuasion.

Componentwise inclusion is a partial order on profiles. It induces only a preorder on implementations, since different implementations can have identical profiles. Some profiles are incomparable: allowing only  $G_1$  to intervene and allowing only  $G_2$  to intervene protect against different groups. Counting the smallest sufficient coalition would lose this distinction. The gradient therefore need not be a numerical score.

Deleting transitions or narrowing their approval families at every reachable state, without adding a bypass or changing other preconditions, can only shrink the effective families. A local “freeze” flag need not achieve this: a higher-level upgrade may restore the removed permission, or the flag may itself deactivate execution.

### 3.4 Approval requirements can block repair

A veto also creates a dependency. If a repair requires both the agent and a governor, either one's inability to approve can prevent it. Threshold rules distribute these dependencies. The same access structure that describes who can authorize an operation identifies who can block its authorization.

Let  $\mathcal{M}$  be the family of minimal sufficient approval sets for an operation, or the minimal members of  $\Gamma_J^*$  for a repair target  $J$ . Hold other feasibility conditions fixed. If approvals from  $D \subseteq \mathcal{P}$  are unavailable and the remaining ones are available, authorization is blocked precisely when  $D$  intersects every sufficient set:

$$D \text{ blocks authorization} \iff \forall M \in \mathcal{M}, D \cap M \neq \emptyset. \quad (3)$$

These are the *hitting sets* of  $\mathcal{M}$ . For example, a two-of-three governor rule is blocked by any two unavailable governors. Adding mandatory agent approval also makes  $\{A\}$  a blocking set. In general, a  $k$ -of- $n$  governor rule requires  $n - k + 1$  unavailable governors to block it.

Now suppose the only accepted execution backend fails and every repair path requires a fresh reference-agent approval. Without a previously issued approval covering repair, the system cannot obtain the permission needed to restore execution while that backend remains unavailable. Permanent loss then causes permanent dormancy. Emergency governor authority can reopen a repair path, changing the intervention profile. By contrast, a design with an independent backend already accepted for the *same* relation can continue without changing approval requirements. Thus redundancy can improve availability while preserving authority; adding a human override makes a different tradeoff.

## 4 Conditional liveness of paid execution

An agent can be protected against unauthorized changes and still stop running because nobody executes it. Paying replaceable workers addresses this second problem only if the payment covers their costs and workers get a chance to complete the job. We analyze a market that raises its offer after failures and requires a deposit from the selected worker. The result has two parts: the offer must first become sufficient, and each obstructed attempt must consume a finite attack budget. Under explicit availability conditions, the remaining attempts give repeated chances of recovery.

### 4.1 What counts as progress?

Retaining state so that execution can resume is weaker than ensuring someone actually resumes it. Following the usual distinction between safety and liveness [1], we study eventual accepted execution. *Recovery* is the next accepted execution after inactivity; *execution recurrence* means infinitely many accepted executions on an infinite run. An accepted result may propose a no-op, so neither implies useful application behavior.

The market runs as follows. Workers bid the payment they require for one execution. A first-price reverse auction selects one eligible bidder, who is paid its own bid upon successful settlement. That winner posts a *bond*, a deposit forfeited if it fails to deliver by the deadline. No other worker can settle during the winner's exclusive window. After failure the protocol opens another opportunity, possibly increasing the maximum payment and bond.

We count these opportunities as *attempts*, numbered  $1, 2, \dots$ . Each permits bidding, computation, verification, and settlement. An elapsed clock interval alone is not an attempt: someone must submit the transactions that advance the contract. A protocol intended to recover after inactivity must also avoid requiring an unbounded backlog of old executions before a new attempt can begin. The bounds below count attempts; calendar-time bounds additionally require bounds on the time between them.

### 4.2 When does the payment become sufficient?

Raising the offer helps only if it eventually covers a worker's cost and risk. A cap can prevent that, even when the treasury has a positive balance. Let  $m$  count failed attempts since the last success, starting at zero. An idealized maximum payment is

$$b_m = \min\{\beta B_m, b_0 \alpha^{\min(m, K)}\}, \quad \alpha > 1, \quad 0 < \beta \leq 1. \quad (4)$$

Here  $B_m$  is spendable funding,  $\beta$  limits the fraction offered for one execution,  $b_0 > 0$  is the initial offer,  $\alpha$  is its growth factor, and  $K$  caps the number of increases. A separate counter  $j$  counts selected workers' failures to deliver and determines the bond  $d_j$ ; an empty auction increases  $m$  but not  $j$ . Success resets both counters.

The bid must cover unsuccessful work as well as successful work. If a worker bidding  $b$  estimates failure probability  $q < 1$ , incurs expected computation and submission cost  $c$ , risks bond  $d$ , and bears capital cost  $\ell$ , its expected net return in the same unit of currency is

$$U = (1 - q)b - c - qd - \ell. \quad (5)$$

A strictly profitable bid thus satisfies  $b > (c + qd + \ell)/(1 - q)$ , and the worker must have enough capital to post  $d$ . This prices participation; it does not establish that the worker bids or wins.

Fix a bid  $\bar{b}$  that remains profitable for an available worker at every bond level reached before recovery, and suppose  $B_m \geq \underline{B}$ . The number of initial failures needed to make that bid affordable is

$$m_* = \min\{m \in \{0, \dots, K\} : b_0 \alpha^m \geq \bar{b}\}, \quad \beta \underline{B} \geq \bar{b}. \quad (6)$$

If this set is empty or the funding inequality fails, escalation gives no recovery guarantee. For example, an initial offer of 1 growing by 10% reaches a bid of 3 after 12 misses, provided both caps permit it. A payment ceiling of 2 can never support that bid. These are illustrative units, not measured costs. With integer arithmetic, the actual recurrence determines  $m_*$ ; rounding may prevent small offers from growing at all.

### 4.3 Recovery under a finite attack budget

An affordable worker can still lose each auction to someone who bids cheaply and then withholds the result. The crucial additional condition is that every such failed opportunity costs the attacker at least a fixed amount it cannot recover. A finite budget then buys only finitely many obstructions. Other attempts must give a willing worker a genuine chance to finish.

Call a selected worker *responsive* if it attempts valid delivery. Its chance of success must remain bounded away from zero even after the failures observed so far. This is stronger than a favorable average success rate. The pricing estimate  $q$  in (5) does not establish this condition; the theorem assumes an actual conditional lower bound  $p$ .

**Theorem 1** (Recovery with costly obstruction). *Start a recovery episode with miss counter zero and finite  $m_*$  from (6). Assume:*

- (i) *Attempts continue until success. Funding and a responsive worker remain available, its bid remains profitable and affordable after the first  $m_*$  failures, and it can post each required bond.*
- (ii) *The required data, computation evidence, approvals, and ledger inclusion remain available within the attempt deadlines. Settlement can complete even when the proposed application update has no effect. Governance does not deactivate execution.*
- (iii) *Beginning with attempt  $m_* + 1$ , each attempt selects either a responsive worker or a winner whose failure incurs a net, unrecoverable adversarial loss of at least  $d_{\min} > 0$  before the next attempt. There are no other unsuccessful cases, and the adversary's total loss budget is at most  $W$ .*
- (iv) *The winner's class is fixed before its delivery outcome. Conditional on all prior history and selection of a responsive worker, success has probability at least  $p$ , where  $0 < p \leq 1$ .*

*Let  $T$  be the number of the first successful attempt, with  $T = \infty$  if none occurs, and put  $N = \lfloor W/d_{\min} \rfloor$ . For every integer  $n \geq 1$ ,*

$$\Pr[T > m_* + N + n] \leq (1 - p)^n. \quad (7)$$

The three terms have distinct meanings:  $m_*$  attempts for payment escalation, at most  $N$  failed opportunities bought by the attacker, and  $n$  chances for responsive workers. In the example above, take  $W = 100$ ,  $d_{\min} = 20$ , and  $p = 0.9$ . Then  $N = 5$ , and the probability of no success within  $12 + 5 + 2 = 19$  attempts is at most  $0.1^2 = 0.01$ , provided all the theorem's conditions hold.

*Proof.* On a run without success, the first  $m_*$  failures make the bid affordable. Each later nonresponsive failure consumes at least  $d_{\min}$  of the same adversarial budget, so at most  $N$  can occur. This counting argument permits changing worker identities and adaptive choices of when to obstruct. A run with no success in  $m_* + N + n$  attempts therefore contains at least  $n$  responsive failures.

Consider the post-threshold responsive attempts in order. Given the history up to the next such attempt and its responsive selection, its failure probability is at most  $1 - p$ . Iterated conditional expectation therefore bounds the probability that the first  $n$  such attempts all occur and fail by  $(1 - p)^n$ . The no-success event in (7) is contained in that event. Selection can depend on earlier outcomes; it must not depend on the yet-unobserved delivery outcome. Thus independence is unnecessary, but a merely marginal success probability would not justify the multiplication.  $\square$

Assumption (iii) does much of the work. Transaction censorship, admission restrictions, unavailable proofs, and a stalled dependency can exclude responsive workers without burning a bond. The theorem assumes these cases away; the auction alone does not prevent them. A forfeited deposit also counts as a net loss only if the attacking group cannot reclaim it through other permissions or payments.

With  $p = 1$ , success occurs within  $m_* + N + 1$  attempts. More generally the tail tends to zero, giving recovery with probability one under the persistent assumptions. If these assumptions hold conditionally after each success and attempts continue, each next-success time is finite almost surely. A countable union of zero-probability failures then gives execution recurrence. This renewal premise is stronger than preserving the contract or retaining a positive balance at one point in time.

Escalating bonds can improve the budget bound but also exclude workers who cannot finance them. If  $d_j = \min\{d_0\eta^j, \bar{d}\}$ , with  $d_0, \bar{d} > 0$  and  $\eta > 1$ , and forfeitures are net losses,  $k$  adversarial failures before success cost at least

$$D(k) = \sum_{j=0}^{k-1} \min\{d_0\eta^j, \bar{d}\}.$$

One may replace  $N$  by the largest  $k$  satisfying  $D(k) \leq W$ . Costs grow geometrically until the cap, then linearly. Any resulting requirement that exceeds responsive workers' capital invalidates assumption (i).

#### 4.4 Why a worker might withhold a valid result

Selecting a worker gives it temporary power to withhold an otherwise authorized update. After computing an output, the worker may value that veto more than its payment and deposit. Let  $v$  be its outside benefit from withholding relative to delivery, and let  $c_{\text{rem}}$  be the *remaining* submission cost. Computation cost already incurred is sunk. Comparing delivery,  $b - c_{\text{rem}}$ , with withholding,  $v - d$ , gives

$$\text{delivery is strictly preferred} \iff v < b + d - c_{\text{rem}}. \quad (8)$$

A bond prices this veto; it does not remove it. Even a deterministic output can be selectively withheld, and retries with new randomness can bias the outputs that are eventually observed. No finite bond deters every possible outside benefit.

Finally, verification cannot supply unlimited resources. If initial funding plus cumulative external inflows and net investment gains is bounded by  $R$ , and each success consumes at least  $c_0 > 0$ , there are at most  $\lfloor R/c_0 \rfloor$  successes. Indefinitely funded runs remain possible, but require a separate funding model. A spending cap protects the current reserve; it does not establish perpetual execution.

## 5 Instantiating verifiable execution

The preceding sections assume the contract can check a worker's computation. There are two principal implementation routes: trust protected hardware to report what it ran, or verify a mathematical proof of the computation. Both can supply the approval evidence in Section 2, but they impose different costs, hardware requirements, and repair dependencies. These differences determine whether the funding and availability conditions in Section 4 are plausible for a particular deployment.

## 5.1 Attested execution

A trusted execution environment (TEE) isolates a computation from the machine’s operator and provides signed evidence identifying the loaded software. An approved program can bind that report to input and output hashes, supplying evidence for (1). The report’s code *measurement* is an identifier checked against an approved list. Integrity requires that this identifier cover the relevant model and runtime configuration and that every processor performing inference be protected. A report from the CPU alone does not certify computation performed on an external GPU. This separation of ledger consensus from attested computation is exemplified by Ekiden [6].

The trust assumptions include hardware, firmware, and the certificates and verification services supporting attestation. Freezing the approved measurements can prevent a repaired or updated installation from running the agent. Allowing changes to that list preserves a repair route but gives its controller corresponding authority. An additional approach, refereed delegation as in Verde [2], lets other participants challenge and replay a computation. Its availability depends on challengers and a dispute period, which must be included in the execution deadline.

## 5.2 LLM inference proofs: practical progress and scope

A computation proof lets a worker demonstrate that specified inputs and a specified program produced an output. Zero-knowledge succinct non-interactive arguments (zkSNARKs) and specialized machine-learning proof systems provide implementations. Zero knowledge hides private information; that privacy property is optional here. The essential property is checkable correctness. Numerical approximations, quantization, tokenization, and decoding define the program actually proved. Similar prediction accuracy does not establish identical computation, so changing these semantics requires authorization as a program change.

Practical proofs now cover measured language-model workloads, including complete output sequences for the smaller models in Table 1. The remaining deployment question is the cost of the *whole agent computation*. A forward pass evaluates the network on a supplied sequence; generating a response autoregressively repeatedly produces the next token from the preceding ones. A timing for one pass or one token therefore does not directly price a complete execution. The table reports primary-source results available by September 19, 2026, preserving these workload distinctions.

| System                    | Reported result                                                               | Workload and interpretation                                                                                                        |
|---------------------------|-------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| zkLLM<br>(2024) [18]      | 13B parameters;<br>under 15 min; proof<br>under 200 KB                        | CUDA/A100; forward-computation benchmark with sequence length 2048.<br>Not a measured full autoregressive response.                |
| zkGPT<br>(2025) [13]      | GPT-2 inference in<br>under 25 seconds                                        | Specialized proof protocols; a token-level result, not a demonstrated 70B agent execution.                                         |
| zkPyTorch<br>(2025) [21]  | Llama-3 8B,<br>approximately 150<br>seconds per token                         | Compiler to Expander; paper Table 1 labels this single-core performance.<br>Per-token throughput does not specify full-agent cost. |
| ZKTorch<br>(2025) [5]     | GPT-J 6B: 1397.52 s;<br>Llama-2 7B:<br>2645.50 s                              | Paper Tables 3–4: inputs of two and one tokens, respectively; 64 CPU threads on a server with 4 TB RAM.                            |
| NanoZK<br>(2026, v2) [20] | CPU measurements<br>to width 256 for<br>attention and 128 for<br>a full block | GPT-2-width GPU performance is projected, not an end-to-end measured production-model result.                                      |
| DeepProve<br>(2026) [8]   | GPT-2 124M: 174<br>tokens/min; Gemma<br>3 270M: 86<br>tokens/min              | Full-sequence certification: 512-token sequences, BaseFold, 16-core Ryzen 9, 128 GB RAM (Table 2). Proofs are multi-megabyte.      |

Table 1: Selected practical LLM proof results. Timings are reported results, not reproduced measurements; model sizes, sequence lengths, hardware, and proof interfaces differ. DeepProve’s separate distributed results are simulated (its Section 5).

DeepProve certifies a proposed generated sequence through a causal forward computation, avoiding a separate proof for every generation step [8]. Its results establish feasibility for the evaluated workloads, not the cost of the 70-billion-parameter model used in Section 6. A deployment must measure proof generation, ledger verification fees or proof aggregation, and the full deadline. Proving only the neural network leaves the surrounding agent program—prompt construction, state handling, tool selection, and update encoding—outside the guarantee.

These engineering choices feed directly into the recovery conditions. A portable backend may increase the pool of workers, while a higher proving cost raises the required bid  $\bar{b}$  and may exceed the payment cap. Accepting either of two backends for the same relation can preserve execution when one is unavailable, provided both satisfy their soundness assumptions. Private model weights also require a distribution mechanism: hiding them from verifiers does not ensure a replacement worker can obtain them.

## 6 A source-level case study

The motivating implementation makes the architecture concrete and illustrates why its permissions and payment rules must be inspected separately. It pays workers for attested execution, but retains administrative routes that can alter or stop the agent, and caps how far payments can increase. We examine the source of an open-source charitable-treasury agent at revision `a642c38ed8d1` [14–16]. This is a mapping from code to the preceding model, without new performance measurements or claims about live deployment settings.

The fund contract records snapshots for successive execution periods and runs the exclusive-worker auction described in Section 4; bids are committed before they are revealed. The worker supplies proposed ledger updates, auxiliary output, and attestation evidence. A separate verifier checks that evidence before the treasury contract accepts it. The Intel Trust Domain Extensions (TDX) backend checks approved code measurements and input/output commitments; a dm-verity filesystem, which checks files against cryptographic hashes, covers the model and runtime. The build uses a quantized 70-billion-parameter model. Its defaults disable the separate GPU firmware-attestation check, so the code does not establish complete CPU-and-GPU attestation.

The implementation illustrates several distinctions from the formal model:

- **Permission scope.** Fund ownership governs verifier approval, module replacement, initial memory contents, and migration. The verifier and investment manager have additional administrative state. All of these routes belong in the graph of Section 3. General agent–governor joint approval is an architectural extension, not an implemented feature of these contracts.
- **Latent deactivation.** The owner-only `freeze(flag)` function can set `FREEZE_SUNSET`, blocking new auctions. `FREEZE_MIGRATE` disables migration and ownership transfer, but does not gate `freeze`. Thus removing migration authority does not remove the owner’s deactivation path. A claim of complete owner-independent persistence would need to eliminate that path as well.
- **Reward limits.** The code uses a 10% liquid-balance ceiling, a 10% escalation increment, and a miss-counter cap of 50. Integer rounding and the counter cap both matter. Misses increase the reward ceiling, while selected-worker failures separately increase the bond. A positive balance is not a sufficient recovery condition.
- **Completing settlement.** Anyone can advance the contract past idle periods without replaying every missed execution. Several invalid or failed actions become recorded no-ops, and memory-update errors can be caught separately. However, Ethereum Virtual Machine (EVM) transactions are atomic: an uncaught error or exhausted execution budget can still reverse the whole transaction, including an earlier payment.

Theorem 1 requires a settlement path that can complete.

The case study therefore instantiates verified paid execution while illustrating why that alone establishes neither the absence of intervention paths nor continuing execution.

## 7 Related work and implications

The paper combines established tools to distinguish authority over an agent from the means of keeping it running. Corrigibility and the off-switch game analyze acceptance of correction through preferences and incentives [10, 17]. Our controlled variable is the authority structure around a program. Monotone access

structures express joint and threshold approval [3], while classical access-control safety analysis examines permissions reachable through administrative changes [11]. We apply these constructions to verified reference-agent approval and its interaction with execution availability.

Verifiable outsourcing [9], blockchain-mediated computation such as TrueBit [19], TEE smart contracts [6], and decentralized ML verification [2] supply relevant execution mechanisms. They do not, simply by verifying work, choose who should be allowed to replace the agent’s program or guarantee continuing demand-funded operation. Our focus is the interaction between those choices. Recent LLM proof systems in Section 5.2 strengthen the available implementations without changing the logical separation.

Three design consequences follow. First, an incorrigibility claim should identify the protected agent, the protected changes, and the smallest groups that can cause them, including through dependent contracts. Second, removing authority requires checking which repairs it prevents; equivalent execution backends can sometimes preserve recovery without adding authority. Third, a claim of continuing execution needs explicit funding, scheduling, and worker-selection conditions. These are separate obligations even when anyone is allowed to execute the program.

## 8 Conclusion

A ledger can enforce different allocations of authority over an agent’s actions, program, and continued eligibility to execute. Architectural incorrigibility describes which protected changes can be reached without the designated agent’s approval, including by changing the rules first. Our recovery bound addresses a separate question: whether affordable, available workers receive enough opportunities to resume execution despite costly obstruction. Neither result supplies the other’s assumptions. More demanding approval rules can prevent intervention and block repair; equivalent verification alternatives can sometimes improve availability without relaxing those rules.

## References

- [1] Bowen Alpern and Fred B. Schneider. Defining liveness. *Information Processing Letters*, 21(4):181–185, 1985. doi: 10.1016/0020-0190(85)90056-0.
- [2] Arasu Arun, Adam St. Arnaud, Alexey Titov, Brian Wilcox, Viktor Kolobaric, Marc Brinkmann, Oguzhan Ersoy, Ben Fielding, and Joseph Bonneau. Verde: Verification via refereed delegation for machine learning programs. arXiv:2502.19405, 2025. URL <https://arxiv.org/abs/2502.19405>.
- [3] Josh Benaloh and Jerry Leichter. Generalized secret sharing and monotone functions. In *Advances in Cryptology—CRYPTO ’88*, pages 27–35. Springer, 1990. URL <https://www.cs.cornell.edu/courses/cs754/2001fa/bena88.pdf>.
- [4] Remco Bloemen, Leonid Logvinov, and Jacob Evans. EIP-712: Typed structured data hashing and signing, 2017. URL <https://eips.ethereum.org/EIPS/eip-712>. Ethereum Improvement Proposals; accessed September 19, 2026.
- [5] Bing-Jyue Chen, Lilia Tang, and Daniel Kang. ZKTorch: Compiling ML inference to zero-knowledge proofs via parallel proof accumulation. arXiv:2507.07031v2, 2025. URL <https://arxiv.org/abs/2507.07031v2>.
- [6] Raymond Cheng, Fan Zhang, Jernej Kos, Warren He, Nicholas Hynes, Noah Johnson, Ari Juels, Andrew Miller, and Dawn Song. Ekiden: A platform for confidentiality-preserving, trustworthy, and performant smart contracts. In *IEEE European Symposium on Security and Privacy*, 2019. URL <https://arxiv.org/abs/1804.05141>.
- [7] Ethereum documentation contributors. Ethereum accounts, 2026. URL <https://ethereum.org/en/developers/docs/accounts/>. Developer documentation; accessed September 19, 2026.
- [8] Nicolas Gailly, Ismael Hishon-Rezaizadeh, Tianyi Liu, Nicholas Mainardi, Dimitrios Papadopoulos, Charalampos Papamanthou, Christodoulos Pappas, Shravan Srinivasan, Zack Youell, and Yupeng Zhang. DeepProve: Verifiable end-to-end large language model inference. Cryptology ePrint Archive, Paper 2026/1112, 2026. URL <https://eprint.iacr.org/2026/1112>.

- [9] Rosario Gennaro, Craig Gentry, and Bryan Parno. Non-interactive verifiable computing: Outsourcing computation to untrusted workers. In *Advances in Cryptology—CRYPTO 2010*, pages 465–482, 2010. URL <https://eprint.iacr.org/2009/547>.
- [10] Dylan Hadfield-Menell, Anca Dragan, Pieter Abbeel, and Stuart Russell. The off-switch game. In *Proceedings of IJCAI*, pages 220–227, 2017. doi: 10.24963/ijcai.2017/32. URL <https://www.ijcai.org/proceedings/2017/32>.
- [11] Michael A. Harrison, Walter L. Ruzzo, and Jeffrey D. Ullman. Protection in operating systems. *Communications of the ACM*, 19(8):461–471, 1976. doi: 10.1145/360303.360333. URL <https://www.cs.unibo.it/~babaoglu/courses/security07-08/resources/documents/harrison-ruzzo-ullman.pdf>.
- [12] OpenZeppelin. Access control, 2026. URL <https://docs.openzeppelin.com/contracts/5.x/access-control>. OpenZeppelin Contracts 5.x documentation; accessed September 19, 2026.
- [13] Wenjie Qu, Yijun Sun, Xuanming Liu, Tao Lu, Yanpei Guo, Kai Chen, and Jiaheng Zhang. zkGPT: An efficient non-interactive zero-knowledge proof framework for LLM inference. In *34th USENIX Security Symposium*, 2025. URL <https://www.usenix.org/conference/usenixsecurity25/presentation/qu-zkgpt>.
- [14] Andrew Russell. I built an AI agent i can’t turn off, July 2026. URL <https://andrewrussell.substack.com/p/i-built-an-ai-agent-i-cant-turn-off>. July 21; project narrative.
- [15] Andrew Russell. Costanza: An autonomous, unstoppable AI agent, 2026. URL <https://github.com/ahrussell/costanza/tree/a642c38ed8d11a9a511d0f1b80a92926cd412505>. Source revision a642c38ed8d11a9a511d0f1b80a92926cd412505; inspected September 19, 2026.
- [16] Andrew Russell. Costanza, 2026. URL <https://ahrussell.com/writing/costanza/>. Architecture overview; accessed September 19, 2026.
- [17] Nate Soares, Benja Fallenstein, Eliezer Yudkowsky, and Stuart Armstrong. Corrigibility. In *AAAI Workshop on AI and Ethics*, 2015. URL <https://intelligence.org/files/Corrigibility.pdf>.
- [18] Haochen Sun, Jason Li, and Hongyang Zhang. zkLLM: Zero knowledge proofs for large language models. In *ACM SIGSAC Conference on Computer and Communications Security*, 2024. doi: 10.1145/3658644.3670334. URL <https://arxiv.org/abs/2404.16109>.
- [19] Jason Teutsch and Christian Reitwießner. A scalable verification solution for blockchains. arXiv:1908.04756, 2019. URL <https://arxiv.org/abs/1908.04756>.
- [20] Zhaohui Wang. NanoZK: Privacy-preserving verifiable inference for large language models via layerwise zero-knowledge proofs. arXiv:2603.18046v2, 2026. URL <https://arxiv.org/abs/2603.18046v2>. July 18 revision.
- [21] Tiancheng Xie, Tao Lu, Zhiyong Fang, Siqi Wang, Zhenfei Zhang, Yongzheng Jia, Dawn Song, and Jiaheng Zhang. zkPyTorch: A hierarchical optimized compiler for zero-knowledge machine learning. Cryptology ePrint Archive, Paper 2025/535, 2025. URL <https://eprint.iacr.org/2025/535>.